

Event-Driven Order Fulfillment System

System Integration – Exam Synopsis (F2026)

1. Introduction	1
2. Problem Statement.....	1
3. Methodology	1
3.1. Theoretical Foundation	1
3.2. Implementation	2
3.3. Hypothesis.....	2
4. Architecture Overview.....	3
5. Synchronous vs Asynchronous communication	4
6. Idempotency and Duplicate messages.....	5
7. Failure Handling, Retries, and Compensation	5
8. Compensating Transactions and the Saga pattern.	5
9. Customer Experience	6
10. Practical Demonstration.	6
11. Conclusion	7

1. Introduction

When a customer places an order on a website, a lot of things need to happen. Stock must be reserved, payment must be authorized, a warehouse must be instructed to pick items, a shipping label must be created, and the customer must be notified. In a monolithic approach which is a synchronous system, these steps are chained in sequence. The HTTP request that places the order waits for all of them to complete before returning a response.

Which could be the following:

- a failure in any single step, for example shipping label API is slow can cause the entire request to fail or time out, even if payment was already charged
- Service becomes tightly coupled, and the Order service must know about every downstream system and call each directly.

- The system cannot be scaled independently.

2. Problem Statement

The question here that we try to answer is: How to design a system where one customer action reliably triggers many independent services, without any of them knowing about each other and without a customer waiting for all of them to finish.

3. Methodology

3.1. Theoretical Foundation

Order fulfillment is not a single transaction, it is a business process that crosses organizational boundaries. Finance must authorize payment, Logistics must reserve stock and coordinate picking, and Customer Relations must keep the customer informed. Each department operates independently and has its own responsibilities and failure tolerance.

The order moves through defined states that carry business meaning:

Pending: the customer's request is received and validated. The business has accepted the order but is not yet committed to fulfilling it.

Confirmed: stock is reserved and payment authorized. The business has now made a binding commitment to the customer.

Processing: the warehouse has the picking instruction, and the shipping label exists. Physical fulfillment is underway.

Cancelled: the business could not fulfill the order. The reason must be recorded and communicated clearly.

Two business rules govern the process and must be preserved by the architecture:

Stock reservation must precede payment authorization. There is no business justification for charging a customer for an item that cannot be delivered. If stock is unavailable, the order is cancelled without involving the payment provider.

Customer notification is not in the critical path. A failed email must never cancel an order. Notification is a best-effort communication, not a fulfillment commitment.

The architecture in section 3 exists to enforce these rules reliably across independent services, not as a technical preference, but because the business process requires it.

3.2. Implementation

Why RabbitMQ over Kafka: RabbitMQ was chosen because this system needs task queues with per-consumer acknowledgment and dead-letter routing. Exactly the model RabbitMQ is designed for. Kafka's strength is durable log replay and high-throughput stream processing which this system does not require. Kafka would also introduce additional complexity for no benefit at this scale. Tools such as MassTransit abstract the broker so switching to Kafka later is straightforward if throughput requires change.

Each service has its own PostgreSQL database and communicates only through events. No service calls another service directly. MassTransit is used to configure consumers and connect them to RabbitMQ queues. Each service runs in a separate process, deployable independently via Docker.

3.3. Hypothesis

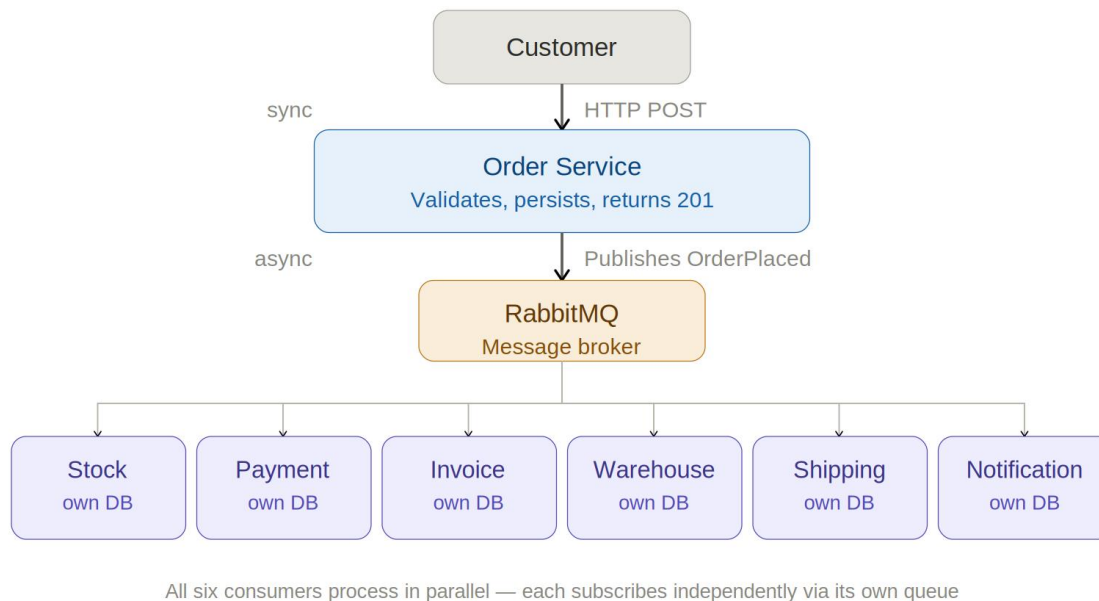
An event-driven publish-subscribe architecture will remove the tight coupling between services and make the system more resilient to individual services failures, compared to a synchronous approach where all steps are chained inside a single HTTP request.

4. Architecture Overview

The system uses an event-driven, publish-subscribe architecture. The Order Service is the single-entry point. When an order is validated and persisted, it publishes one event `OrderPlaced` to a RabbitMQ message broker. Six independent consumer services each subscribe to this event and process it in parallel.

Service	Responsibility
Stock Service	Reserves inventory for ordered items
Payment Service	Sends an authorization request to the payment provider
Invoice Service	Generates a PDF invoice and stores it
Warehouse Service	Creates a picking task for warehouse staff
Shipping Service	Calls a carrier API to generate a shipping label
Notification Service	Sends an order confirmation email or SMS

The diagram below illustrates the event flow. The customer's HTTP POST is the only synchronous call. Everything downstream is asynchronous and non-blocking.



5. Synchronous vs Asynchronous communication

We use both, but boundaries between them are necessary for design decisions.

Synchronous (client -> Order Service)

The initial HTTP request to the Order Service is Synchronous. The client needs to see an immediate response to the order. The order service validates the request, persists in the order with status pending. And returns a 201 response.

Asynchronous (Order Service -> all downstream service)

After the order persists, the order service publishes Order placed and returns. All downstream service processing happens asynchronously in the background.

This means the customer gets a fast response regardless of how long payment auth or label generates takes.

Let's say shipping service is temporarily unavailable, the message waits in the queue and is delivered when the service recovers the order is not lost that way and all the services can be scaled, deployed or restarted independently without affecting each other.

This came with a tradeoff:

The system becomes eventually consistent so right now the data might be wrong but in a moment it will be correct. When the customer receives their 201, their stock is not yet reserved, and their payment is not yet charged. This is acceptable because the window is milliseconds to seconds, and the system is designed to handle failures gracefully.

6. Idempotency and Duplicate messages

Message brokers guarantee at least once delivery – under network failure or consumer crash, the same message may be delivered more than once. A consumer that is not idempotent will be double charge customers or double reserve stock when this happens.

Each consumer is made idempotent using an outbox or deduplication pattern:

- Every message carries a unique MessageId(a UUID tied to the original Order Placed event).
- Before processing, the consumer checks a ProcessedMessages table for this ID.
- If the ID is already present, the message is acknowledged and discarded without re-processing.
- If a new consumer processes it and records the ID atomically in the same database transaction.

The payment Service requires extra care. A payment authorization that is retried against an external provider must carry a stable idempotency key (the OrderId), so the provider deduplicates on its side. Most real payment APIs support this.

7. Failure Handling, Retries, and Compensation

Retry policies

In case a network timeout temporary service unavailability we implement MassTransit

Which supports retrying intervals per consumer. For example, a Service retries up to three times with delays of 1s, 5s, and 30s before giving up.

Dead- letter queues

if a message exhaust all retries, it is moved to a dead letter queue DLQ rather than being discarded. This is monitored separately. Manually inspect failed messages, fix underlying causes, and re-queue them. Nothing is lost.

8. Compensating Transactions and the Saga pattern.

Some failures require more than a retry – they require undoing work that has already succeeded. For example, if Payment fails after Stock has already reserved inventory, the stock reservation must be released.

This is modelled as a saga, a sequence of local transactions coordinated through events. The system uses choreography style: -each service publishes outcome events (Payment Failed, Stock Reserved) and other services react accordingly.

Step	Event Published	Service Reacting	Result
1	OrderPlaced	Stock Service	Reserves Inventory → Publishes StockReserved
2	StockReserved	Payment Service	Attempts charge → card declined → publishes PaymentFailed
3	PaymentFailed	Stock Service	Receives PaymentFailed → releases reservation → publishes StockReleased
4	PaymentFailed	Order Service	Updates order status to Cancelled
5	PaymentFailed	Notification Service	Sends cancellation email to customer

Payment is only invoked after StockReserved is received. This enforces the invariant that no customer is charged for an item that cannot be delivered. The choreography wires this rule into the event sequence rather than into application code.

9. Customer Experience

The happy path for the customer's perspective:

- Customer clicks 'Place order' and receives a 201 and order confirmation page immediately.
- Within seconds get a confirmation email (Notification Service processed the event)
- Also, payment is charged and the invoice is in their account.
- Within a few minutes, warehouses see the picking task; shipping label generated.

When something fails mid-process, our design principle is never leaving the customer in a state where not sure what is happening. Examples:

- Payment fails, the order status update to Cancelled and cancelation email is sent, so the customer sees a clear status.
- If notification Service fails, the order still proceeds as notification not in a critical path. The customer maybe not receive an email, but their order is fulfilled.

10. Practical Demonstration.

We demonstrate in the project the following scenarios

- **Happy path:** place an order and observe all six-consumer processing the event in parallel via the RabbitMQ management UI.
- **Idempotency:** replay the same OrderPlaced message and show that no service processes it twice.
- **Transient failure:** simulate a crash in the Payment Service mid-processing; show the Stock Service release the reservation in response.
- **Dead letter queue:** exhaust all retries on a consumer; show the message appearing in DLQ without being lost.
- **Observability:** filter logs by a single Order to reconstruct the full event trace across all service.
- **Business rule validation:** attempt to place an order for a product with zero stock. Show that the Stock Service publishes StockUnavailable and the order moves to Cancelled without the Payment Service ever being invoked demonstrating that the business rule (no charge without stock) is enforced at the architectural level.

11. Conclusion

The naive synchronous approach causes problems described in the problem statement. The event-driven approach solves all three. The hypothesis is confirmed.

The project demonstrates the core principles of event-driven architecture within a realistic but intentionally scoped system. Building it surfaced with several practical tensions that theory alone does not reveal.

The most significant challenge was idempotency. It is easy to state that consumers must be idempotent but implementing it correctly checking and recording a MessageId atomically within the same database transaction requires care. A non-atomic implementation introduces a race condition where two deliveries of the same message both pass the check before either records the ID.

The saga in choreography style also revealed a trade-off: the logic of the overall business process is distributed across multiple services, making it harder to follow than a single orchestrator would be. For a larger team or a more complex process, an orchestration approach using MassTransit's saga state machine would make the process flow explicitly and auditable in one place.

Given more time, the following improvements would improve the system:

- **Outbox pattern:** the Order Service currently publishes the event and writes to the database as two separate operations. If the service crashes between them, the order persists, but the event is never published. The transactional outbox pattern solves this by writing the event to a database table in the same transaction and publishing from there.
- **Saga orchestration:** replacing choreography with a MassTransit saga state machine would make the order lifecycle states explicit in code and simplify failure recovery.
- **Schema registry:** as the number of event types grows, coordinating contract changes across services becomes difficult. A schema registry enforces compatibility between producers and consumers.
- **Health checks and monitoring:** production readiness would require liveness and readiness endpoints on each service, plus a dashboard aggregating queue depths, consumer lag, and dead-letter queue size.